

Instructions for Lab Submission

1. File Naming and Submission:

- Submit your solution as a **ZIP file** named `lab<number>.<ERP>.zip`, where `<ERP>` is your ERP number. For example, if your ERP number is 12345, the file should be named `lab1_12345.zip`.
- Ensure that the ZIP file contains:
 - C++ source files named as `task<number>.cpp` (e.g., `task1.cpp`).
 - No unnecessary files (such as executables, temporary files, or editor configs).
- Do not submit **TEXT(.txt)** or **EXE(.exe)** files.

2. Code Organization:

- Write each task in its own `.cpp` file (as specified above).
- Ensure that your code is modular and logically separated into functions when specified.

3. Code Neatness:

- Ensure your code is **well-formatted** and easy to read. Use proper **indentation** and **meaningful variable names**.
- Include appropriate **comments** to explain the logic where necessary (especially for functions and complex sections of the code).
- Add clear explanations where required (or specified)** to make the logic of your program easy to follow.

4. Code Compilation and Functionality:

- Ensure your code compiles without errors or warnings. You should be able to compile and run your code on any system with a standard C++ compiler (e.g., GCC, Clang, MSVC).
- Double-check that the functionality of the program meets the requirements outlined in the task.

5. Additional Instructions:

- Do not use any libraries or features that have not been covered in the course material, unless specifically instructed.
- Make sure your code adheres to the principles of good software development (e.g., modular functions, minimal repetition, and no hardcoding).
- If you encounter any issues or require clarifications, reach out before the submission deadline.

6. Deadline:

- Ensure that you submit your lab on time, before the deadline. Late submissions will not be accepted.
- Double-check that the zip file is correctly named and contains all necessary files before submitting.

7. Plagiarism Policy:

- The work submitted must be entirely your own. Any code or content copied from others (including online sources, classmates or AI) will result in an automatic zero for the task.

Lecture Review

1. Arrays

An **array** stores multiple values of the same data type under one variable name.

```
int marks[5] = {72, 85, 61, 90, 77};
```

The array has **5** elements. Array positions are called **indices**, and indexing starts from **0**.

Index	0	1	2	3	4
Value	72	85	61	90	77

Individual elements can be accessed or modified using their index:

```
cout << marks[0];    // 72
cout << marks[3];    // 90

marks[2] = 70;       // changes 61 to 70
```

For an array of size **SIZE**, the valid indices are:

$$0, 1, 2, \dots, \text{SIZE} - 1$$

Accessing an index outside this range can lead to incorrect behavior or a runtime error.

2. Traversing an Array

Traversal means visiting the elements of an array one by one.

A **for** loop is commonly used for traversal:

```
const int SIZE = 5;
int numbers[SIZE] = {4, 7, 2, 9, 6};

for (int i = 0; i < SIZE; i++) {
    cout << numbers[i] << " ";
}
```

The variable **i** represents the current index.

Traversal can also be used to perform calculations:

```
int total = 0;

for (int i = 0; i < SIZE; i++) {
    total += numbers[i];
}
```

Similarly, a loop can be used to find a minimum, maximum, count values, or search for a particular element.

3. Nested Loops

A loop inside another loop is called a **nested loop**.

The outer loop commonly controls the rows, while the inner loop controls the columns.

```
for (int row = 0; row < 3; row++) {
    for (int column = 0; column < 4; column++) {
```

```
        cout << "* ";
    }

    cout << endl;
}
```

Output:

```
* * * *
* * * *
* * * *
```

For every one iteration of the outer loop, the inner loop completes all of its iterations.

4. Two-Dimensional Arrays

A **2D array** stores values in rows and columns.

For example:

```
int marks[3][4] = {
    {72, 85, 61, 90},
    {68, 79, 88, 75},
    {91, 84, 73, 89}
};
```

This array contains **3** rows and **4** columns.

	Column 0	Column 1	Column 2	Column 3
Row 0	72	85	61	90
Row 1	68	79	88	75
Row 2	91	84	73	89

An element is accessed using two indices:

```
cout << marks[0][1];    // 85
cout << marks[2][3];    // 89

marks[1][2] = 95;       // changes 88 to 95
```

The first index represents the **row**, and the second index represents the **column**.

Therefore:

```
array[row][column]
```

5. Traversing a 2D Array

A nested loop is normally used to visit every element of a 2D array.

```
const int ROWS = 3;
const int COLS = 4;

for (int row = 0; row < ROWS; row++) {

    for (int column = 0; column < COLS; column++) {
        cout << marks[row][column] << " ";
    }

    cout << endl;
}
```

The outer loop moves through the rows, while the inner loop moves through the columns of the current row.

The general pattern is:

```
for (int row = 0; row < ROWS; row++) {  
    for (int column = 0; column < COLS; column++) {  
        // process array[row][column]  
    }  
}
```

6. Grids

A **grid** is a collection of cells arranged in rows and columns. A 2D array can be used to represent a grid.

For example:

```
int grid[4][5] = {  
    {1, 0, 0, 1, 0},  
    {0, 1, 1, 0, 0},  
    {1, 1, 0, 0, 1},  
    {0, 0, 1, 1, 0}  
};
```

Each cell can be accessed using:

```
grid[row][column]
```

For example:

```
cout << grid[2][4];    // 1
```

When working with a grid, nested loops allow us to process every cell.

For example, counting all cells containing **1**:

```
int count = 0;  
  
for (int row = 0; row < 4; row++) {  
    for (int column = 0; column < 5; column++) {  
        if (grid[row][column] == 1) {  
            count++;  
        }  
    }  
}
```

7. Processing Rows and Columns

Nested loops can be used to perform calculations for individual rows or columns.

For example, calculating the sum of every row:

```
for (int row = 0; row < ROWS; row++) {  
    int sum = 0;  
    for (int column = 0; column < COLS; column++) {  
        sum += grid[row][column];  
    }  
}
```

```
    }  
  
    cout << "Row " << row << ": " << sum << endl;  
}
```

The position of the variables matters:

- `row` identifies which row is being processed.
- `column` identifies which column is being processed.
- `grid[row][column]` refers to the current cell.

8. Important Reminders

- Array indexing starts from `0`.
- An array of size `SIZE` has valid indices from `0` to `SIZE - 1`.
- A 2D array uses two indices: `array[row][column]`.
- The first index represents the row and the second represents the column.
- For a 2D array with `ROWS` rows and `COLS` columns, valid row indices are `0` to `ROWS - 1`.
- Valid column indices are `0` to `COLS - 1`.
- Use `<`, not `<=`, when traversing array indices.
- Nested loops are commonly used to process 2D arrays and grids.
- The outer loop usually controls rows, while the inner loop controls columns.
- Always make sure that an index stays within the valid range.

Lab Tasks

Task 1: The Broken Selection Sort

A student information system stores the marks of students in one array and their corresponding grade letters in another array. The system must arrange the students according to their marks in descending order.

The following arrays are initially stored:

```
int marks[8] = {72, 91, 65, 88, 54, 79, 96, 83};  
char grades[8] = {'C', 'A', 'D', 'B', 'F', 'C', 'A', 'B'};
```

The position of each grade corresponds to the student whose marks are stored at the same position. Therefore, whenever two marks are exchanged during sorting, their corresponding grade letters must also be exchanged.

The programmer attempted to implement selection sort in descending order, but the program contains several logical and syntactical errors.

Problem Description

The selection sort should process the array from left to right. For every position `i`, the program should find the index of the largest mark in the unsorted portion of the array and place that mark at position `i`.

Whenever two marks are exchanged, the corresponding elements of the `grades` array must be exchanged at exactly the same positions so that the relationship between marks and grades is not lost.

The following program attempts to perform the required sorting:

```
#include <iostream>  
using namespace std;  
  
int main() {  
  
    const int SIZE = 8;  
  
    int marks[SIZE] = {72, 91, 65, 88, 54, 79, 96, 83};  
    char grades[SIZE] = {'C', 'A', 'D', 'B', 'F', 'C', 'A', 'B'};  
  
    cout << "Original Data:" << endl;  
  
    for (int i = 0; i < SIZE; i++) {  
        cout << marks[i] << "(" << grades[i] << ") ";  
    }  
  
    cout << endl;  
  
    for (int i = 0; i < SIZE - 1; i++) {  
  
        int maxIndex = i + 1;  
  
        for (int j = i + 1; j < SIZE; j++) {  
  
            if (marks[j] < marks[maxIndex]) {  
                maxIndex = j;  
            }  
  
        }  
  
        if (maxIndex != i) {  
  
            int temp = marks[i];  
  
            marks[i] = marks[maxIndex];  
  
            marks[maxIndex] = temp;  
  
            char tempGrade = grades[i];  
  
            grades[i] = grades[maxIndex];  
  
            grades[maxIndex] = tempGrade;  
  
        }  
  
    }  
  
}
```

```

        // Add your code here
    }
}

cout << "\nSorted Data:" << endl;

for (int i = 0; i < SIZE; i++) {
    cout << marks[i] << "(" << grades[i] << ") ";
}

cout << endl;

return 0;
}

```

Find and fix all errors in the program so that the marks are sorted in descending order using selection sort.

The corresponding grade letters must move together with their marks. For example, if a mark moves from one position to another, its grade must move to the same position.

Keep the overall structure of the program and use selection sort. Do not use `std::sort`, `std::swap`, or any other built-in sorting function.

Note: The final marks must be in descending order, and every grade must remain associated with its original mark.

Task 2: The Duplicate Record Detector

A university stores the IDs of students registered in two different sections of a course. Some students may have accidentally been registered in both sections.

The system stores the student IDs in two separate arrays:

```

sectionA = {104, 117, 125, 138, 149, 163, 172, 185}
sectionB = {109, 125, 131, 149, 158, 172, 194}

```

The university wants to identify all student IDs that appear in both sections.

Problem Description

For every ID in `sectionB`, the program must determine whether the same ID exists in `sectionA`.

The IDs in `sectionA` are already arranged in ascending order. Because of this, a binary search should be used when checking an ID against `sectionA`.

For comparison, a linear search must also be performed through `sectionA` for every ID in `sectionB`. The program should count the total number of comparisons performed by each searching approach.

The following pseudocode describes the required searching process:

```

SET sectionA = [104, 117, 125, 138, 149, 163, 172, 185]
SET sectionB = [109, 125, 131, 149, 158, 172, 194]

SET SIZE_A = 8
SET SIZE_B = 7

SET binaryComparisons = 0
SET linearComparisons = 0

DISPLAY "Duplicate IDs:"

FOR i FROM 0 TO SIZE_B - 1
    SET target = sectionB[i]

    SET low = 0

```

```

SET high = SIZE_A - 1
SET foundBinary = false

WHILE low < high

    SET mid = (low + high) / 2
    INCREASE binaryComparisons BY 1

    IF sectionA[mid] EQUALS target
        SET foundBinary = true
        STOP THE WHILE LOOP

    ELSE IF sectionA[mid] < target
        SET low = mid + 1

    ELSE
        SET high = mid - 1

    END IF

END WHILE

SET foundLinear = false
FOR j FROM 0 TO SIZE_A - 1

    INCREASE linearComparisons BY 1

    IF sectionA[j] EQUALS target
        SET foundLinear = true
        STOP THE FOR LOOP
    END IF

END FOR

IF foundBinary AND foundLinear
    DISPLAY target
END IF

END FOR

DISPLAY "Binary Search Comparisons: ", binaryComparisons
DISPLAY "Linear Search Comparisons: ", linearComparisons

```

Convert the above pseudocode into a complete C++ program. Your implementation must follow the searching process described in the pseudocode and satisfy all the requirements given below.

Find and fix the logical errors in the binary-search portion of the pseudocode so that every ID in `sectionB` is correctly checked against `sectionA`.

The linear search portion must also correctly determine whether an ID exists. An ID should be displayed only if it is present in both sections.

Do not sort either array and do not use any built-in searching functions.

Note: The program must use both binary search and linear search, and must display the total number of comparisons performed by each approach.

Task 3: The Zeroed Matrix

A building's monitoring system represents different areas of the building using an $m \times n$ integer matrix. Each element $a[i][j]$ stores the operational status of a particular area, where 0 indicates that the area is disabled.

Whenever an area is found to be disabled, its entire row and column must also be marked as disabled to indicate that the

corresponding areas are affected.

Write a program that processes the matrix according to this rule: if $a[i][j]$ is 0, set all elements in row i and column j to 0.

The changes must be based only on the zeroes present in the ****original matrix****. A zero created during the process must not cause any additional rows or columns to be zeroed.

For example, given the following matrix:

1	2	3	4
5	6	7	8
0	9	0	1

The resulting matrix should be:

0	2	0	4
0	6	0	8
0	0	0	0

Use arrays and loops to solve the problem.

Task 4: Square Matrix Multiplication

A scientific application performs calculations on square matrices of the same size. Two $N \times N$ matrices are given by the user and must be multiplied to produce a third $N \times N$ matrix.

For two matrices A and B , each element of the resulting matrix C is calculated by multiplying the corresponding elements from a row of A and a column of B , then adding all of those products.

In mathematical form,

$$C_{ij} = \sum_{k=0}^{N-1} A_{ik} B_{kj}$$

where $0 \leq i, j < N$.

Problem Description

Write a program that first inputs the value of N . The value of N represents the number of rows and columns in both matrices.

The program should then input all elements of matrix A , followed by all elements of matrix B .

The program must calculate the matrix C using the standard matrix multiplication process. Each element of C should be calculated by combining one complete row of A with one complete column of B .

Finally, display matrix C in a properly formatted $N \times N$ grid.

Use nested loops to perform the multiplication. Do not use any built-in matrix multiplication function.

Note: The value of N should be entered by the user, and the program should work for different square matrix sizes.